

AI-FORECAST

Complete Team Study & Viva Defense Guide • Full-Stack Time-Series Analytics

Tech Stack: React 18 + Vite • Tailwind CSS • Recharts • FastAPI (Python 3.10+) • Scikit-learn • Joblib

1. Project in 60 Seconds

AI-FORECAST is an end-to-end full-stack web application designed for historical time-series forecasting. It eliminates the fatal flaw of *temporal data leakage* seen in toy projects by enforcing strict chronological validation splitting (shuffle=False) across all feature engineering, model training, and recursive inference operations.

1. Data Ingestion & Imputation: Handles raw CSV time-series, detects timestamp & numeric target columns, and applies automated imputation (ffill / mean).	2. Temporal Feature Engineering: Constructs autoregressive lags (lag_1, lag_7), rolling trend averages (7-day), and calendar attributes.
3. Baseline & Ensemble Regressors: Trains Linear Regression baseline (interpretable OLS) alongside Random Forest (100 decision trees for non-linear seasonality).	4. Out-of-Sample Leaderboard & Future Forecast: Benchmarks models on a 20% holdout set with MAE/RMSE/R ² , then recursively projects 7–30 days with 95% confidence bands.

2. High-Level System Architecture

The system is organized into a clean decoupled client-server architecture:

Frontend (React 18 + Vite) - Port: http://localhost:5173 - Tailwind CSS Light Neutral Design System - Recharts interactive data visualization - Modular page-by-page state management	API Layer (FastAPI REST) - Port: http://127.0.0.1:8000 - OpenAPI interactive docs at /docs - Pydantic input/output schemas - Modular router endpoints	ML Engine (Scikit-learn) - Chronological train/test partitions - Recursive multi-step forecaster - Error propagation confidence bands - Joblib serialization registry
--	--	--

3. Folder Structure & Key Code Locations

Directory / File	Key Responsibilities & Contents
backend/app/main.py	FastAPI app instance, CORS middleware, routing registration.
backend/app/api/	Endpoints for datasets, preprocessing, analytics, models, evaluation, forecasts.
backend/app/ml/training_service.py	Chronological 80/20 train/validation split logic and Scikit-learn estimator training.
backend/app/ml/forecaster.py	Multi-step recursive autoregressive forecast generator with expanding uncertainty.
backend/app/ml/evaluation.py	Mathematical loss functions: MAE, MSE, RMSE, and R ² calculation.
frontend/src/App.jsx	Main application layout, global dataset/model state, navigation, error boundary.
frontend/src/pages/	The 7 core pages: Dashboard, Datasets, Preprocessing, Analytics, Models, Evaluation, Forecasts, Viva.
frontend/src/charts/	Light-themed Recharts: TimeSeriesChart, ActualVsPredictedChart, ForecastChart.

4. End-to-End Data Pipeline (Step-by-Step)

- Step 1 — Raw Ingestion:** CSV uploaded or sample loaded (120 daily records). Timestamp & numeric targets identified.
- Step 2 — Feature Engineering:** Calendar features (dayofweek, month), autoregressive lags (lag_1, lag_7), and 7-day rolling average.
- Step 3 — Chronological Partition:** Earliest 80% (Days 1–96) assigned to Training; subsequent 20% (Days 97–120) assigned to Test.
- Step 4 — Dual Training:** Fits Linear Regression (intercept calculation) & Random Forest (100 trees, max_depth=10, random_state=42).

- **Step 5 — Holdout Validation:** Evaluates both models on unseen 20% test partition. Lowest RMSE model flagged as 'Recommended'.
- **Step 6 — Recursive Future Forecasting:** Projects future dates by iteratively substituting predicted values back into lag inputs. Uncertainty bounds expand dynamically: $\sigma(h) = RMSE \times \sqrt{(1 + 0.10(h-1))}$.

5. Application Pages & Features Walkthrough

Page Name	User Action / Interface	Behind-the-Scenes Engineering
1. Dashboard	Status overview, 1-click sample load, 5-step workflow overview.	Health-check ping to /health, checks dataset & model registries.
2. Datasets	Upload custom CSV or inspect loaded rows and target columns.	Pandas schema inference, date parsing, missing value identification.
3. Preprocessing	Select imputation strategy (ffill, mean), toggle lag & rolling features.	Computes lag_1, lag_7, rolling_mean_7, outputs audit log.
4. Analytics	Interactive time-series line chart, 7/14/30-day moving average windows.	Calculates Min, Max, Mean, Latest metrics and rolling time-series.
5. Models	Select Linear Regression or Random Forest; adjust sliders and train.	Strict 80/20 chronological split, trains Scikit-learn estimator, serializes.
6. Evaluation	Comparison leaderboard (MAE, RMSE, R²), actual vs predicted chart.	Predicts holdout test set, measures residual errors, recommends lowest RMSE.
7. Forecasts	Pick horizon (7, 14, 30d), confidence level (80/95/99%), export CSV.	Recursive multi-step forecasting loop, error propagation confidence bands.
8. Viva & Docs	Examiner Q&A; accordions, formula reference cards, 5-min demo script.	Self-contained project defense hub for college viva presentation.

6. Core Machine Learning Concepts (Viva Essentials)

Q1: Why must we use chronological splitting instead of random K-Fold CV?

In time series, data points have temporal auto-correlation. If you randomly shuffle observations, future points (t+5) leak into the training set while past points (t+1) end up in the test set. This creates **look-ahead bias (data leakage)**, giving fake high accuracy during development but total failure in production. We strictly enforce shuffle=False (earliest 80% train, latest 20% test).

Q2: Why compare Linear Regression against Random Forest?

Linear Regression provides an instant (<20ms), perfectly interpretable statistical baseline with zero risk of tree overfitting. **Random Forest** uses an ensemble of 100 decision trees to capture non-linear seasonal cycles, weekend multipliers, and holiday demand spikes without requiring data stationarity.

Q3: What is the mathematical difference between MAE and RMSE? Why prioritize RMSE?

MAE = $(1/n) \sum |y - \hat{y}|$. Measures average error magnitude linearly. **RMSE** = $\sqrt{(1/n) \sum (y - \hat{y})^2}$. By squaring errors before taking the square root, RMSE heavily penalizes large blunders. In supply chain and demand planning, missing a huge demand spike causes severe stockouts; hence RMSE is our primary ranking metric.

Q4: Can the R² score be negative on the holdout test set?

Yes. $R^2 = 1 - (SS_{res} / SS_{tot})$. On unseen test data, if an overfitted model's residual sum of squares exceeds the total variance of the actual ground truth, R² becomes negative. This proves the model performs worse than simply predicting the historical mean (y_{mean}) for every future point.

Q5: How does the recursive forecasting engine work for future dates?

Because future ground truth is unavailable, at step t+1 the model predicts $\hat{y}_{t+1}$. It then recursively injects $\hat{y}_{t+1}$ back into the buffer as the lag_1 input for step t+2, repeating iteratively across the entire forecast horizon H.

Q6: Why do confidence intervals widen over time?

Because predictions are recursively recycled as inputs, forecasting uncertainty compounds at every step. We model standard error dynamically as: $\sigma(h) = \text{RMSE_val} \times \sqrt{(1 + 0.10 \times (h - 1))}$. As horizon step h increases, the uncertainty cone realistically expands.

7. How to Run & Develop Locally

Terminal 1 – Backend (FastAPI):
cd backend && .\venv\Scripts\Activate.ps1
python -m uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload
(API live at http://127.0.0.1:8000 • Interactive docs at /docs)

Terminal 2 – Frontend (Vite):
cd frontend && npm run dev
(Frontend live at http://localhost:5173)

Terminal 3 – Run Automated Tests:
cd backend && .\venv\Scripts\python -m pytest
(All 26 unit and integration tests pass with 100% success rate)

8. Team Member Presentation / Viva Defense Roles

Team Member	Assigned Topics & Presentation Sequence
Member 1 (Architecture & Data)	- Explains the problem statement and the dangers of temporal data leakage. - Demonstrates the Dashboard and Datasets page. - Explains how FastAPI REST backend and React frontend communicate asynchronously.
Member 2 (Preprocessing & ML)	- Demonstrates Preprocessing: explains why we engineer lag_1, lag_7, and rolling_mean_7. - Demonstrates Models: explains the 80/20 chronological split with shuffle=False. - Compares Linear Regression vs. Random Forest hyperparameters and execution time.
Member 3 (Evaluation & Forecasts)	- Demonstrates Evaluation: explains MAE, RMSE, and R², and why the lowest RMSE model wins. - Demonstrates Forecasts: generates 14-day projection, explains recursive forecasting and confidence intervals, and exports the CSV. - Answers theoretical examiner questions using the built-in Viva & Docs tab.